

ADVANCE PROBLEM PROGRAMMING

SLO 2: Modular Programming

1. Break a Program into Multiple Reusable Methods in Java

Modular programming divides a program into smaller, independent, and reusable methods. Each method performs a specific task.

```
public class Main {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
  
    static int multiply(int a, int b) {  
        return a * b;  
    }  
  
    static void display(int result) {  
        System.out.println("Result = " + result);  
    }  
  
    public static void main(String[] args) {  
        int sum = add(10, 20);  
        int product = multiply(10, 20);  
  
        display(sum);  
        display(product);  
    }  
}
```

```
}
```

Output:

Result = 30

Result = 200

Here, add(), multiply(), and display() are separate reusable modules.

2. Write a Java Program That Uses a Main Method and Two Helper Methods

```
public class Main {  
  
    static int square(int n) {  
        return n * n;  
    }  
  
    static void display(int result) {  
        System.out.println("Square = " + result);  
    }  
  
    public static void main(String[] args) {  
        int number = 5;  
  
        int result = square(number);  
        display(result);  
    }  
}
```

Output:

Square = 25

Here, main() controls the program, while square() and display() are helper methods that perform specific tasks.

3. Explain How Java Supports Code Reuse via Static Methods

A **static method** belongs to the class rather than to a particular object. It can be called directly using the class name without creating an object.

```
class Calculator {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        System.out.println(Calculator.add(10, 20));  
        System.out.println(Calculator.add(30, 40));  
    }  
}
```

Output:

30

70

The add() method is defined only once but can be called multiple times with different values. This provides **code reuse**.